

Reti di Calcolatori

Elena Giunta / Canale A-L / 2024-2025

Introduzione

La nostra epoca è caratterizzata dalla condivisione di informazioni attraverso reti di calcolatori interconnessi, che permettono lo scambio di dati tramite vari mezzi (cavi, fibre ottiche, satelliti, ecc.). Una rete di calcolatori differisce da un sistema distribuito, dove i computer appaiono come un unico sistema coerente grazie a un software, chiamato middleware. Nelle reti, invece, gli utenti vedono le macchine reali senza un'apparenza unificata.

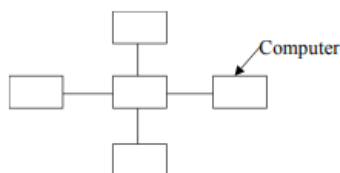
Un sistema di comunicazione è composto da due parti fondamentali: il **mezzo fisico** (hardware) e la **struttura logica** (software), che rappresenta l'insieme di protocolli e regole che governano la comunicazione tra dispositivi.

Classificazione delle reti

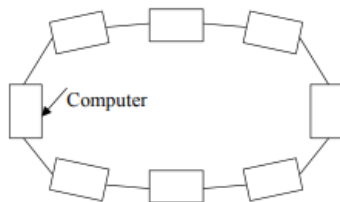
- **RETI LOCALI (Local Area Network, LAN):** reti di estensione limitata (1 km) che consentono il collegamento di dispositivi collocati nello stesso edificio o in edifici adiacenti. La velocità di trasmissione delle reti locali è molto più elevata di quella delle reti geografiche.
- **RETI METROPOLITANE (Metropolitan Area Network, MAN):** reti di dimensione tale da consentire il collegamento di dispositivi collocati nella stessa area urbana (10 km).
- **RETI GEOGRAFICHE (Wide Area Network, WAN):** reti di ampia dimensione impiegate per il collegamento di dispositivi diffusi in un'ampia area geografica (1.000 km).

Topologie di rete

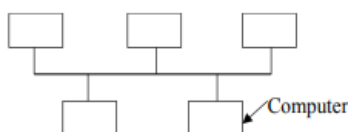
Topologia a stella



Topologia ad anello



Topologia a bus



Diversi tipi di rete

Reti Point-to-Point (P2P): **Ogni coppia di dispositivi è connessa direttamente** tramite un canale dedicato. La comunicazione avviene **solo tra i due nodi connessi**. Può essere fisica (cavo diretto) o logica (VPN, tunnel).

Reti Broadcast: **Un solo mittente trasmette il messaggio a tutti i dispositivi della rete**. Ogni dispositivo riceve il messaggio, ma solo il destinatario lo elabora. È usata in reti condivise, come Ethernet e Wi-Fi. Rischio di congestione della rete se troppi dispositivi trasmettono contemporaneamente.

Che cos'è un protocollo?

La comunicazione tra due calcolatori richiede che vengano fissate delle regole che descrivano il modo in cui la comunicazione tra di essi deve avvenire. Un protocollo di comunicazione è l'insieme di queste regole. I protocolli specificano i formati dei dati, la struttura dei pacchetti e la velocità di trasmissione.

Nella comunicazione ciascun calcolatore fa uso di un insieme di protocolli, ognuno dedicato ad un particolare aspetto della comunicazione.

Essi definiscono il formato, l'ordine dei messaggi mandati e ricevuti nel network.

Facciamo un'analogia con i "protocolli" umani e quelli di rete per capire meglio cosa sono questi ultimi.

Protocolli umani

I protocolli umani sono regole sociali che guidano le interazioni tra persone. Definiscono:

1. **Cosa dire:** messaggi specifici (es. "Che ore sono?").
2. **Quando dirlo:** sequenza logica (es. domanda → risposta).
3. **Cosa fare:** azioni intraprese in base ai messaggi o eventi (es. rispondere all'ora).

Protocolli di rete

I protocolli di rete sono regole tecniche che governano la comunicazione tra dispositivi (computer, router, server, ecc.). Definiscono:

1. **Formato dei messaggi:** struttura dei dati (es. pacchetti TCP/IP).
2. **Ordine dei messaggi:** sequenza di operazioni.
3. **Azioni intraprese:** gestione di errori, ritrasmissioni, conferme.

Struttura a strati (o livelli) delle reti

Le reti sono organizzate in **strati** per ridurre la complessità. Ogni strato offre servizi allo strato superiore, nascondendo i dettagli implementativi. Questo approccio crea una sorta di **macchina virtuale** per ogni livello.

Comunicazione tra strati

1. **Comunicazione orizzontale (tra pari):**
 - Lo strato **n** di un computer comunica con lo strato **n** di un altro computer utilizzando un **protocollo di strato n**.
 - I **pari (peer)** sono le entità (processi, hardware, ecc.) che risiedono sullo stesso strato in dispositivi diversi e comunicano tramite protocolli.
2. **Comunicazione verticale (tra strati):**
 - Lo strato **n** non comunica direttamente con il suo pari remoto, ma passa i dati allo strato sottostante (**n-1**), che a sua volta li passa al livello inferiore, fino ad arrivare al mezzo fisico.
 - All'arrivo, i dati risalgono gli strati fino a raggiungere il livello corrispondente sul dispositivo di destinazione.

Interfacce

Tra ogni coppia di strati c'è un'**interfaccia** che facilita la comunicazione tra di essi.

- Le interfacce espongono le operazioni necessarie per propagare i messaggi verso l'alto o verso il basso.
- Grazie alle interfacce, è possibile modificare un singolo strato senza dover cambiare l'intera architettura.

Mezzo fisico

- Sotto l'ultimo strato si trova il **mezzo fisico** (es. cavi in rame, fibre ottiche, WiFi), che permette la trasmissione effettiva dei dati.

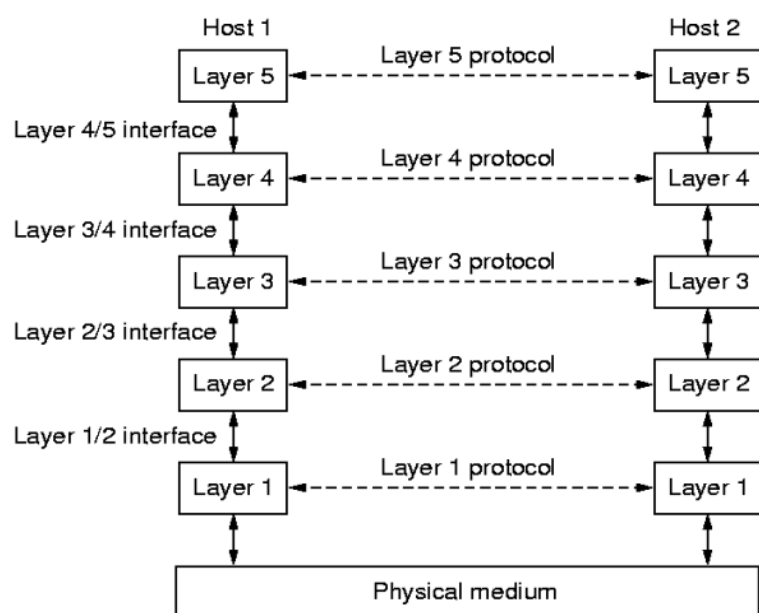
Architettura di rete

- L'insieme di strati e protocolli costituisce l'**architettura di rete**.

- I dettagli di implementazione e le interfacce non fanno parte dell'architettura. Non è necessario che le interfacce dei computer siano tutte uguali, purché i protocolli standard vengano rispettati.

Pila dei protocolli

- I protocolli sono organizzati in una **pila dei protocolli**, che facilita la progettazione e l'implementazione delle reti. La pila di protocolli di Internet consiste di cinque livelli: fisico, collegamento, rete, trasporto e applicazione.



I problemi della progettazione a livelli

Per garantire una corretta comunicazione, alcuni aspetti cruciali devono essere considerati durante la progettazione di un sistema di comunicazione a livelli.

1. **Indirizzamento:** Ogni dispositivo che comunica in una rete deve avere un indirizzo univoco, che consenta di identificare chiaramente il destinatario del messaggio. Gli indirizzi IP e le porte sono esempi di come viene realizzato l'indirizzamento nelle comunicazioni di rete.
2. **Controllo degli errori:** Le comunicazioni tramite canali fisici non sono esenti da errori, che possono essere causati da interferenze, rumore o problemi hardware. Per garantire l'affidabilità, vengono utilizzati meccanismi di rilevamento e correzione degli errori. Entrambi i dispositivi coinvolti devono essere d'accordo sul metodo da utilizzare per il controllo.
3. **Controllo di flusso:** Quando il mittente invia dati più rapidamente di quanto il destinatario riesca a riceverli, si verifica un sovraccarico del canale di comunicazione.
4. **Routing o istradamento:** In una rete complessa, esistono spesso più percorsi tra il mittente e il destinatario. L'algoritmo di routing ha il compito di determinare il percorso

più efficiente, che può essere basato su criteri come il costo, la distanza o la congestione della rete.

5. **Lunghezza dei messaggi:** Alcuni processi possono avere limitazioni riguardo alla dimensione dei messaggi che possono essere trasmessi. Se un messaggio è troppo grande, deve essere suddiviso in pacchetti più piccoli, che vengono inviati separatamente e poi riassemblati una volta giunti a destinazione.
 6. **Ordine di invio dei messaggi:** In alcune reti, i pacchetti possono arrivare fuori ordine. Bisogna quindi riordinare i pacchetti correttamente una volta arrivati a destinazione, in modo che il destinatario possa ricostruire il messaggio originale.
 7. **Direzione di invio:** In alcune situazioni, le comunicazioni sono unidirezionali (ad esempio, il broadcasting televisivo), mentre in altre sono bidirezionali (come le comunicazioni tra client e server). Quando la comunicazione è unidirezionale, non è necessario gestire il flusso di risposta, mentre in una comunicazione bidirezionale devono essere previsti meccanismi di gestione delle richieste e delle risposte.
 8. **Quantitativo di canali di comunicazione:** Se una rete ha più canali di comunicazione, può assegnare un canale diverso a ogni connessione. Tuttavia, spesso è più efficiente condividere lo stesso canale tra più connessioni, usando tecniche di multiplexing. Ad esempio, con il **multiplexing a divisione di tempo (TDM)**, più segnali vengono trasmessi su un unico canale alternandosi in momenti diversi. Questo permette di ottimizzare l'uso del canale e gestire più trasmissioni contemporaneamente senza interferenze.
-

Servizi Orientati alla Connessione

Funzionano come un **sistema telefonico**, dove per poter comunicare con una macchina remota è necessario prima stabilire una connessione. Questo processo include una negoziazione per definire i parametri della comunicazione.

- Una volta stabilita, la trasmissione è **affidabile**: i dati arrivano nell'ordine corretto e vengono ritrasmessi se persi o corrotti.
- Un esempio è **TCP (Transmission Control Protocol)** usato per navigazione web, email e trasferimento file.

Servizi Senza Connessione

Funzionano come un **sistema postale**, ogni messaggio contiene l'indirizzo completo del destinatario e viene instradato indipendentemente, senza una connessione persistente tra mittente e destinatario. In questo caso, i pacchetti vengono trattati **indipendentemente** e non c'è una **conferma** che essi arrivino correttamente o nell'ordine giusto.

- Un servizio di questo genere viene detto datagram ed ha una minore affidabilità.

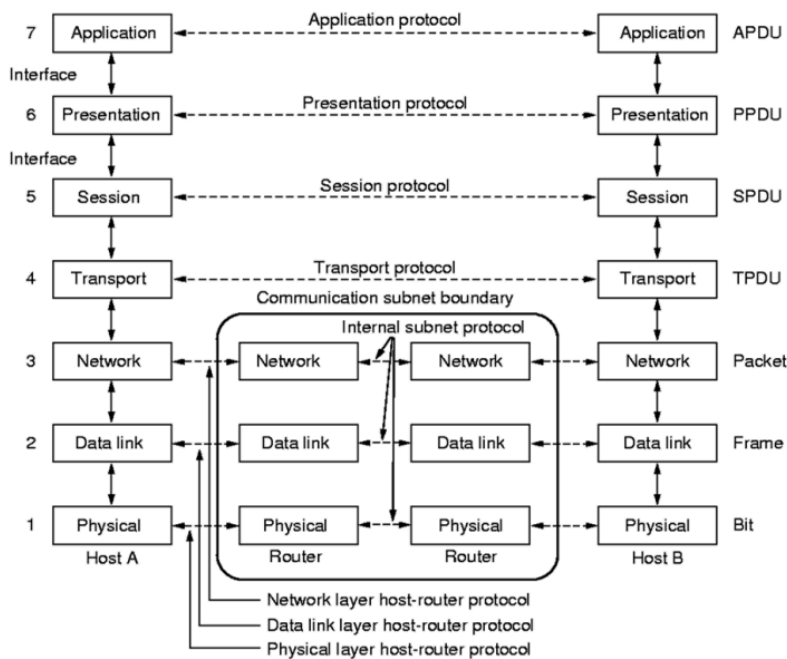
- **UDP (User Datagram Protocol)** ne è un esempio; viene utilizzato in applicazioni che richiedono bassa latenza, come streaming in tempo reale. In questi casi, la perdita di alcuni pacchetti è accettabile, poiché la priorità è ridurre al minimo il ritardo.
 - In alcuni casi si usano meccanismi come il **Request-Reply** o il **Datagram con Connessione**, che forniscono conferma di ricezione senza mantenere una connessione continua.
-

Il Modello OSI

Il **modello OSI (Open Systems Interconnection)** è un modello di riferimento per le comunicazioni di rete, sviluppato dall'ISO (International Organization for Standardization) negli anni '80. Anche se i protocolli OSI non sono utilizzati, il modello rimane importante per comprendere come funziona la comunicazione tra dispositivi di rete.

Il modello è suddiviso in **7 strati**

1. **Strato Fisico:** Si occupa della trasmissione dei **bit** grezzi attraverso il mezzo fisico; (interfacce meccaniche o elettriche)
2. **Strato Data Link (Collegamento Dati):** Gestisce la trasmissione affidabile dei dati tra due dispositivi direttamente connessi. Si occupa del **rilevamento degli errori**, del **controllo del flusso** e dell'**accesso al canale** di comunicazione. Suddivide i dati in **frame** e gestisce gli indirizzi MAC.
3. **Strato Network (Rete):** Si occupa del **routing** (instradamento dei pacchetti) tra dispositivi su reti diverse. Gestisce la qualità del servizio e il controllo della congestione. Utilizza gli indirizzi IP per identificare in modo univoco i dispositivi.
4. **Strato Transport (Trasporto):** Garantisce una trasmissione **affidabile** o **non affidabile** dei dati tra dispositivi. Divide i dati in **segmenti** e ne garantisce l'ordine e l'integrità.
5. **Strato Session (Sessione):** Gestisce l'inizio, il mantenimento e la terminazione delle sessioni di comunicazione tra due dispositivi. Sincronizza le operazioni tra client e server. Tiene traccia dei turni di trasmissione e ricezione, gestione dei token.
6. **Strato Presentazione:** Si occupa della sintassi e della semantica dell'informazione. Gestisce le strutture dati astratte che permettono a due computer con diverse architetture di comunicare. (Si occupa della **conversione dei dati** in un formato comprensibile da mittente e destinatario. Eseguisce la compressione e la crittografia dei dati.)
7. **Strato Application (Applicazione):** È lo strato più vicino all'utente, che fornisce servizi di rete alle applicazioni. Include protocolli per email, web, trasferimento file, ecc.



Perchè il modello OSI non è più adottato?

1. **Tempismo sbagliato** – Quando OSI è stato sviluppato, **TCP/IP** era già ampiamente adottato, quindi nessuno ha investito nel nuovo standard.
2. **Progettazione complessa** – La struttura del modello OSI risultava **pesante e difficile da implementare**.
3. **Implementazioni lente** – A causa della **complessità del modello**, gli aggiornamenti erano lenti, mentre **TCP/IP** si evolveva rapidamente.
4. **Mancato supporto politico** – Essendo visto come un progetto governativo, **l'industria lo ha rifiutato**, preferendo soluzioni più pratiche.

In che senso progetto governativo? Il modello OSI è stato sviluppato dall'**ISO (International Organization for Standardization)**, un ente internazionale di standardizzazione, con il supporto di vari governi e organizzazioni pubbliche.

Al contrario, **TCP/IP** è nato in ambito accademico e militare, sviluppato negli Stati Uniti con finanziamenti del **Dipartimento della Difesa (DARPA)** per garantire comunicazioni affidabili tra computer in caso di guerra.

Quando OSI fu proposto come standard globale, molte aziende lo **percepirono come un'imposizione burocratica**, piuttosto che un'innovazione pratica. Quindi, il termine "progetto governativo" si riferisce al fatto che OSI era visto come **un'iniziativa imposta dall'alto**, mentre TCP/IP veniva adottato spontaneamente dall'industria.

ARPANET: Il Precursore di Internet

ARPANET, finanziata dal **Dipartimento della Difesa USA (DoD)**, è stata la prima rete geografica (**WAN**) e il precursore di **Internet**. Il progetto nacque dalla necessità di una rete resistente ai guasti, superando le limitazioni della tradizionale rete telefonica.

L'idea si basava sulla **commutazione di pacchetto**, che permetteva di suddividere i dati in piccoli pacchetti trasmessi separatamente e poi riassemblati a destinazione. Questo garantiva una maggiore affidabilità: in caso di guasto di un nodo, i pacchetti venivano instradati su percorsi alternativi.

Nel **1969**, ARPANET collegò i primi quattro nodi presso le università **UCLA, Stanford, UCSB e Utah**. La rete utilizzava minicomputer chiamati **IMP (Interface Message Processor)** per gestire il traffico tra gli host e assicurare la trasmissione efficiente dei dati.

ARPANET fu fondamentale per lo sviluppo del modello **TCP/IP** (1983), il protocollo che ha poi dato vita a **Internet**.

I pacchetti di dati nella rete

Prima di parlare del modello TCP/IP, capiamo com'è fatto un pacchetto di dati.

Nelle reti, la **trasmissione delle informazioni** avviene attraverso l'invio di **pacchetti di dati**. Questi pacchetti sono unità discrete di informazione in cui un messaggio più grande viene suddiviso per essere trasmesso in modo efficiente sulla rete.

Header (Intestazione)

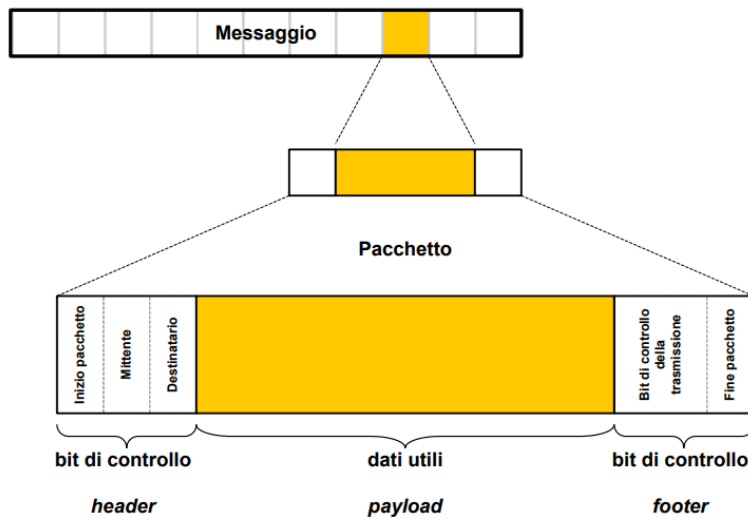
- Contiene **informazioni di controllo** necessarie per l'instradamento del pacchetto.
- Include campi come:
 - **Inizio pacchetto**: indica l'inizio del pacchetto.
 - **Mittente**: identifica chi ha inviato il pacchetto.
 - **Destinatario**: specifica il destinatario finale.

Payload (Dati Utili)

- È la parte centrale del pacchetto, che contiene i **dati veri e propri** (cioè l'informazione trasmessa).
- Il payload rappresenta la porzione di dati utile per l'applicazione finale, come un frammento di un file o un pezzo di un messaggio di posta elettronica.

Footer (Coda o Controllo di Fine Pacchetto)

- Contiene **bit di controllo** per verificare che il pacchetto sia stato ricevuto correttamente.
- Include:
 - **Bit di controllo della trasmissione**: consentono al ricevitore di verificare che i dati non siano stati alterati durante il trasporto.
 - **Fine pacchetto**: indica il termine del pacchetto.



Commutazione di circuito e commutazione di pacchetto

Le reti di comunicazione utilizzano due principali tecniche di commutazione: la commutazione di circuito (circuit switching) e la commutazione di pacchetto (packet switching).

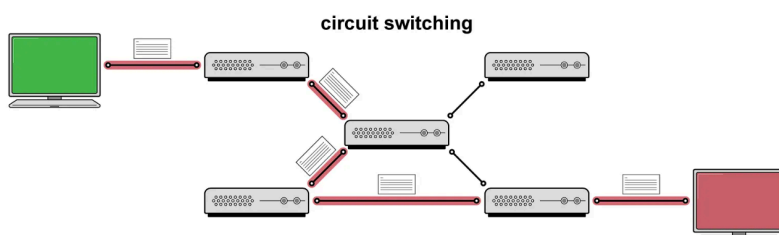
1. Commutazione di Circuito

Nella **commutazione di circuito**, viene stabilito un collegamento fisso tra mittente e destinatario prima dell'inizio della comunicazione. Questo percorso rimane riservato per tutta la durata dello scambio di dati, garantendo una trasmissione continua e senza interruzioni.

Le fasi principali della commutazione di circuito sono:

1. **Creazione del collegamento** tra i due utenti, vengono allocate risorse come la larghezza di banda e le linee di comunicazione.
2. **Scambio di informazioni** attraverso il percorso riservato.
3. **Abbattimento del collegamento** al termine della comunicazione.

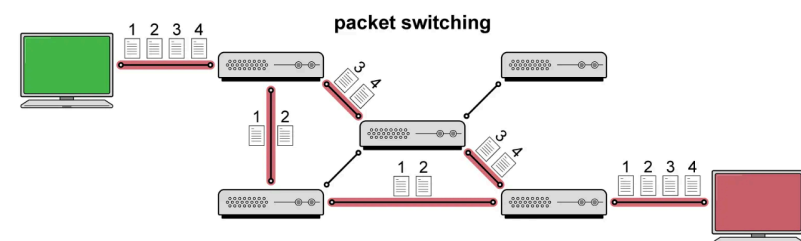
- **Esempio:** Telefonata tradizionale (rete PSTN) → Una volta che la chiamata è avviata, il percorso tra i due telefoni rimane costante fino alla fine della conversazione.
- **Vantaggi:** Connessione stabile e senza ritardi variabili.
- **Svantaggi:** Spreco di risorse se la comunicazione ha momenti di inattività, poiché il collegamento rimane comunque occupato.



2. Commutazione di pacchetto

Nella **commutazione di pacchetto**, i dati vengono suddivisi in piccoli pacchetti che viaggiano autonomamente attraverso la rete senza la necessità di un collegamento fisso tra mittente e destinatario.

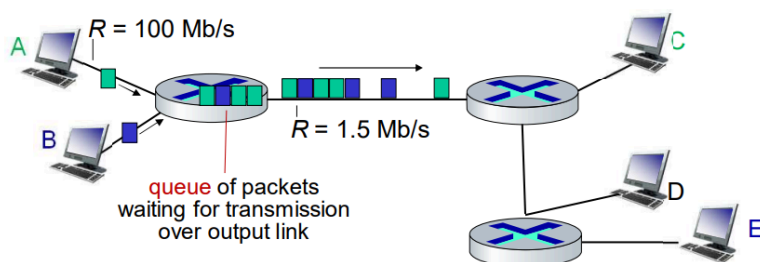
La **commutazione di pacchetto** si suddivide in due principali metodologie: **commutazione a datagrammi** e **commutazione a circuito virtuale**.



Commutazione di Pacchetto (Datagram)

Nella **commutazione di pacchetti datagram**, i dati vengono suddivisi in piccoli pacchetti che viaggiano indipendentemente attraverso la rete, senza un percorso predefinito. I router o gli switch decidono il percorso per ogni pacchetto in modo dinamico in base alle condizioni di rete correnti come congestione e disponibilità. Poiché i pacchetti possono seguire percorsi diversi in base alle condizioni della rete (come congestione o guasti), devono essere riassemblati una volta arrivati a destinazione.

- **Esempio:** Internet → Quando invii un'email o visiti un sito web, i dati vengono suddivisi in pacchetti che possono attraversare la rete su percorsi differenti prima di raggiungere la destinazione.
- **Vantaggi:** Uso più efficiente della rete, le **risorse non vengono riservate**, ma sono condivise dinamicamente tra più utenti.
- **Svantaggi:** Possibile ritardo nella trasmissione se la rete è congestionata o se i pacchetti arrivano fuori ordine.



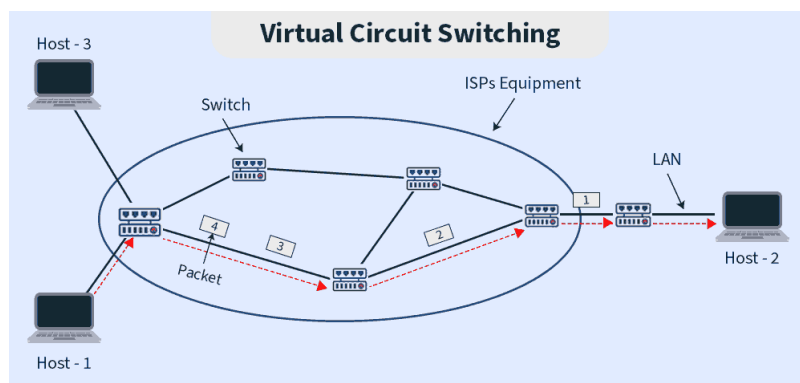
Commutazione di pacchetti a circuito virtuale

Il **circuito virtuale (VC)** è una tecnica che combina aspetti della **commutazione di circuito** e della **commutazione di pacchetto**. Pur utilizzando una rete a commutazione di pacchetto, viene stabilito un **percorso logico** tra mittente e destinatario prima dell'inizio

della trasmissione. Tutti i pacchetti seguiranno questo percorso, garantendo maggiore ordine e affidabilità rispetto ai datagrammi, evitando riassettaggio complesso.

- **Condivisione dinamica delle risorse:** A differenza della commutazione di circuito, le risorse fisiche (es. banda) sono condivise tra più circuiti virtuali.
- **Esempio:** ATM (Asynchronous Transfer Mode) e alcune VPN.
- **Vantaggi:** Maggiore affidabilità rispetto alla commutazione a datagrammi, perché i pacchetti arrivano nell'ordine corretto.
- **Svantaggi:** Minore flessibilità rispetto ai datagrammi, perché il percorso scelto deve essere mantenuto per tutta la comunicazione.

È importante distinguere il circuito virtuale dalla commutazione di circuito tradizionale. Mentre quest'ultima stabilisce una connessione fisica dedicata per l'intera sessione di comunicazione, il circuito virtuale opera su una rete a commutazione di pacchetto e condivide le risorse di rete in modo dinamico in base alla domanda. Le risorse non sono allocate esclusivamente per la connessione, consentendo un utilizzo più efficiente della rete.



Il modello TCP/IP

Il **modello TCP/IP** è formato da 4 strati ed è più semplice e pratico rispetto al modello OSI. Inoltre, è meno rigido nella separazione tra servizi, interfacce e protocolli.

1. Strato Internet (IP)

Gestisce la **trasmissione dei pacchetti** tra dispositivi su diverse reti.

- I pacchetti viaggiano **in modo indipendente**, senza connessione fissa tra mittente e destinatario.
- Il **protocollo IP (Internet Protocol)** si occupa dell'instradamento dei pacchetti.

2. Strato Trasporto (TCP/UDP)

Permette la comunicazione tra applicazioni sui diversi host. Sono stati definiti due protocolli di trasporto end-to-end:

- **TCP** (Transmission Control Protocol)

Affidabile, garantisce l'ordine dei dati e la loro completa trasmissione, è usato per email, siti web, trasferimenti file.

- **UDP** (User Datagram Protocol)

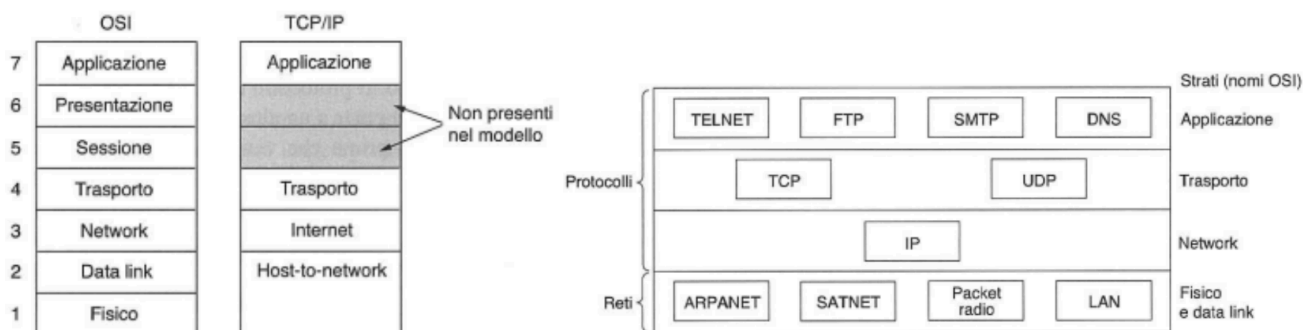
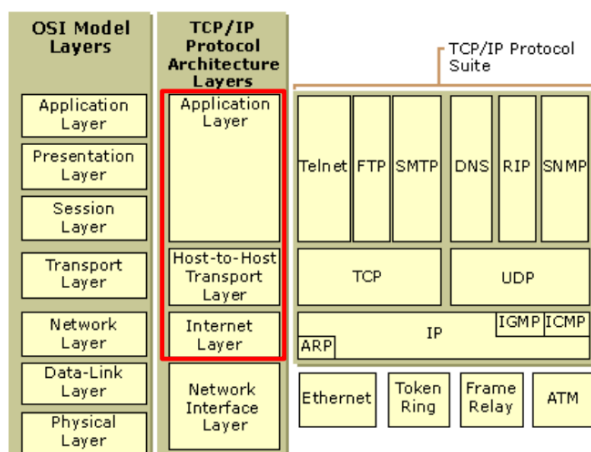
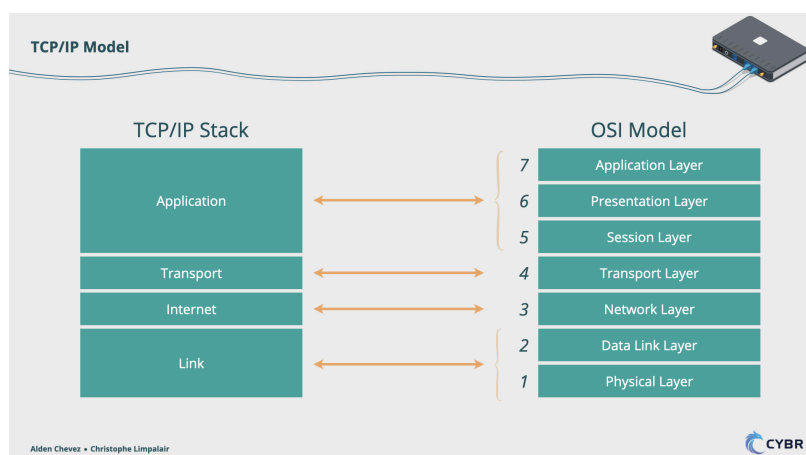
Più veloce, ma non garantisce l'affidabilità, è usato per streaming video/audio, giochi online.

3. Strato Applicazione

Contiene tutti i protocolli di livello superiore, come: TELNET (terminale virtuale), FTP (trasferimento file), SMTP (email), DNS (corrispondenza tra nome degli host e indirizzi di rete) e molti altri.

4. Strato Host-to-Network

Definisce il collegamento fisico tra un host e la rete (Ethernet, Wi-Fi, 4G, ecc.).



Tuttavia, anche TCP/IP ha alcuni **difetti strutturali**, nonostante la sua diffusione:

1. **Mancanza di separazione tra servizio e protocollo**, il che rende il modello meno chiaro.
2. **Non è adatto per descrivere protocolli diversi da TCP/IP** (es. Bluetooth).
3. **Lo strato host-to-network non è un vero strato**, ma solo un'interfaccia tra rete e data link.
4. **Non distingue tra strato fisico e data link**, che di norma sono due cose distinte e separate.
5. Modello troppo radicato e **difficile da adattare a nuove tecnologie**
6. **Il modello teorico è quasi inesistente**, ma i protocolli sono di uso comune.

Confronto tra i modelli di riferimento OSI e TCP/IP

Entrambi i modelli sono basati su una pila di protocolli stratificati, dove ogni livello ha una funzione specifica e fornisce servizi agli strati superiori.

Il modello OSI introduce tre concetti chiave:

1. **Servizi** → Descrivono *cosa* fa ogni strato, senza specificare come.
2. **Interfacce** → Definiscono *come* gli strati comunicano tra loro.
3. **Protocolli** → Regole di comunicazione tra entità dello stesso strato su dispositivi diversi.

TCP/IP non distingue nettamente questi aspetti, risultando più flessibile ma meno modulare.

A differenza di OSI, TCP/IP è stato costruito intorno a protocolli esistenti, risultando meno modulare ma più efficiente e diffuso.

Un'importante differenza riguarda la gestione della comunicazione:

- **OSI** supporta sia comunicazioni orientate alla connessione che non nello strato di rete, mentre nel trasporto prevede solo connessioni affidabili.
- **TCP/IP** utilizza solo una comunicazione senza connessione a livello di rete (IP), ma nel trasporto supporta sia connessioni affidabili (TCP) che non affidabili (UDP).

Caratteristica	Modello OSI	Modello TCP/IP
Origine	Modello teorico, nato prima dei protocolli.	Basato su protocolli esistenti (creato a posteriori).
Numero di strati	7 strati.	4 strati.
Flessibilità	Generale e adattabile a nuove tecnologie.	Più pratico e ottimizzato per Internet.
Indipendenza dai protocolli	Separazione netta tra strati → facile sostituzione di protocolli.	Più vincolato ai protocolli esistenti.
Strato Network	Supporta sia comunicazione orientata che non alla connessione.	Solo comunicazione senza connessione.
Strato Trasporto	Supporta solo connessioni affidabili (orientate alla connessione).	Supporta sia TCP (affidabile) che UDP (non affidabile).
Uso nella pratica	Più teorico e didattico, usato come riferimento.	Direttamente implementato nelle reti moderne.

Strati OSI (7 livelli)	Strati TCP/IP (4 livelli)
Applicazione	Applicazione
Presentazione	(Incluso nell'Applicazione)
Sessione	(Incluso nell'Applicazione)
Trasporto	Trasporto
Rete	Internet
Data Link	(Incluso in Accesso alla Rete)
Fisico	Accesso alla Rete

Livello Applicativo

Protocolli di Livello Applicazione e Requisiti delle Applicazioni

Un **protocollo di livello applicazione** definisce le modalità di comunicazione tra processi tramite:

- **Tipi di messaggi:** richieste, risposte.
- **Sintassi:** struttura e delimitazione dei campi nei messaggi.
- **Semantica:** significato delle informazioni nei messaggi.
- **Regole di comunicazione:** tempi e modalità di invio e ricezione dei messaggi.

Tipologie di protocolli

I protocolli di livello applicazione possono essere **open** o **proprietary**:

- **Open protocols:** Definiti in **RFC** e accessibili pubblicamente. Consentono **interoperabilità** tra diversi sistemi (es. **HTTP**, **SMTP**).
- **Proprietary protocols:** Non pubblici e di proprietà di aziende (es. **Skype**).

Ogni applicazione ha le sue esigenze, e i protocolli di rete vengono progettati proprio per soddisfare questi requisiti. La scelta tra un protocollo **affidabile ma più lento** e uno **veloce**

ma meno preciso dipende dal tipo di servizio che si vuole offrire.

Come Comunicano i Processi nelle Reti

In un sistema informatico, i **processi** (programmi in esecuzione nell'host) hanno bisogno di un canale affidabile e bidirezionale per comunicare tra loro. Se si trovano sullo **stesso computer**, usano meccanismi di **comunicazione interprocesso (IPC)**, gestiti dal sistema operativo. Ma quando i processi sono su **macchine diverse**, la comunicazione avviene tramite **messaggi scambiati in rete**: un processo invia dati e l'altro li riceve, rispondendo se necessario.

Nelle applicazioni di rete, i processi si dividono generalmente in **client** e **server**:

- Il **client** è il dispositivo o il programma che **inizia la comunicazione**. I client **contattano** e **comunicano** con il server per richiedere servizi o dati. Possono essere **connessi in modo intermittente** e generalmente hanno **indirizzi IP dinamici**.
- Il **server**, è un dispositivo o un programma che è sempre in **attesa di richieste** da parte dei client. Il server è un **host sempre attivo (always-on)** con un **indirizzo IP permanente (statico)**, che gli consente di essere facilmente raggiungibile da qualsiasi client.

L'**architettura di un'applicazione** definisce come essa viene organizzata sui diversi sistemi periferici e si basa generalmente su una delle due principali architetture di rete utilizzate oggi:

- **L'architettura client-server**
- **L'architettura peer-to-peer (P2P)**

Queste due architetture determinano il modo in cui i dispositivi comunicano tra loro e gestiscono lo scambio di dati all'interno di una rete.

Architettura Client-Server

L'**architettura client-server** si basa su un host sempre attivo, chiamato **server**, che fornisce servizi e risponde alle richieste di numerosi altri host, detti **client**.

Il processo dà inizio alla comunicazione, il processo server attende di essere contattato.

Un aspetto fondamentale di questo modello è che **i client non comunicano direttamente tra loro**.

- Un esempio di questa architettura è **un'applicazione web**, in cui un **web server** è sempre attivo e pronto a rispondere alle richieste dei browser in esecuzione sui client. Quando un client invia una richiesta per ottenere un oggetto (ad esempio una pagina web o un file), il server elabora la richiesta e risponde inviando i dati richiesti. I browser degli utenti non si scambiano dati tra loro, ma interagiscono esclusivamente con il server.

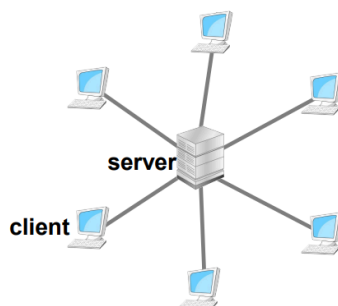
Per garantire la comunicazione, il server è un host sempre attivo e dispone di un **indirizzo IP statico e noto**, che consente ai client di contattarlo in qualsiasi momento, inviandogli pacchetti di dati (i client invece hanno IP dinamico). Grazie a questa configurazione, i servizi client-server possono garantire una **gestione centralizzata delle richieste e delle risorse**.

Tra le applicazioni più note basate su un'architettura client-server troviamo:

- **Il Web** (navigazione attraverso i browser)
- **FTP** (File Transfer Protocol, utilizzato per il trasferimento di file)
- **Telnet** (protocollo per il controllo remoto di computer)
- **La posta elettronica** (gestione centralizzata dei messaggi e-mail)

Un potenziale limite dell'architettura client-server è la sua **scalabilità**. Se un'applicazione ha un numero elevato di utenti, un singolo server potrebbe non essere in grado di gestire tutte le richieste contemporaneamente.

Per risolvere questo problema, nelle architetture client-server si adottano spesso **data center**, ovvero grandi infrastrutture che ospitano numerosi host, i quali lavorano insieme per creare un **server virtuale** più potente.



client/server

Architettura Peer-to-Peer (P2P)

L'**architettura peer-to-peer (P2P)** si distingue dal modello client-server perché non prevede un server centrale: i **peer** (cioè gli host connessi alla rete) comunicano direttamente tra loro. In questa architettura, i dispositivi come **computer fissi, portatili e altri dispositivi degli utenti** sono in grado di scambiarsi dati senza la necessità di un'infrastruttura centralizzata, come accade nel tradizionale modello client-server, dove le richieste passano sempre attraverso un server.

****Vantaggi:**

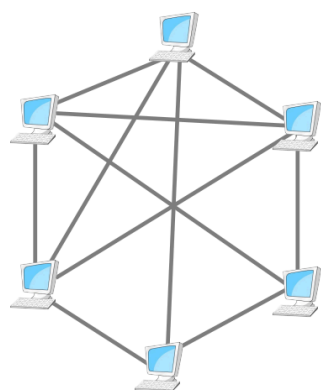
- L'architettura P2P è adatta a scenari con **alta intensità di traffico**: ne è esempio la **condivisione di file tramite BitTorrent**, dove gli utenti scaricano e caricano file senza dipendere da un server centrale.
- Maggiore **scalabilità**: all'aumentare del numero di utenti, **aumenta anche la capacità del sistema**, poiché ogni nuovo peer contribuisce alla rete fornendo risorse aggiuntive.

- **Costo ridotto:** a differenza dell'architettura client-server, che richiede un'infrastruttura server centralizzata, P2P distribuisce il carico di lavoro tra i peer.

Svantaggi:

- **Sicurezza:** Poiché i peer comunicano direttamente tra loro, la rete può essere vulnerabile ad attacchi informatici e alla diffusione di software dannoso.
- **Prestazioni:** A seconda della connessione e della disponibilità degli altri peer, la velocità di trasferimento dei dati può variare.
- **Affidabilità:** Non essendoci un server centrale che garantisce la disponibilità costante delle risorse, il sistema può risultare meno stabile rispetto a un'infrastruttura client-server.

Il modello P2P continua a essere una soluzione efficiente ed economica per molte applicazioni moderne, in particolare quelle che richiedono una distribuzione **decentralizzata** delle risorse.



peer-to-peer

Sockets

I processi in rete comunicano inviando e ricevendo messaggi attraverso un meccanismo fondamentale chiamato **socket**. Le socket sono gli **intermediari** tra il **livello applicazione** e di **trasporto** nello stack TCP/IP, infatti la funzione dei socket è quella di **indirizzamento dei processi**.

Ogni scambio di dati coinvolge **due socket**:

1. **Socket mittente** → Da cui parte il messaggio.
2. **Socket destinatario** → A cui arriva il messaggio.

Funzionamento Pratico:

1. L'applicazione mittente "spinge" i dati nella socket
2. Il protocollo di trasporto si occupa della consegna
3. L'applicazione destinataria "preleva" i dati dalla sua socket

Attraverso il socket, client e server sono in grado di “parlarsi” e di scambiare dati a due vie, ricevendo e trasmettendo dati entrambi. Questo processo di comunicazione bidirezionale è ciò che rende possibile l’interazione tra applicazioni e servizi online.

- Le socket fungono da **API (Interfaccia di Programmazione)** tra l’applicazione e la rete, essendo l’interfaccia mediante la quale le applicazioni sono costruite.

Controllo del Programmatore

Lo sviluppatore dell’applicazione ha pieno controllo sulla parte applicativa del socket, ma ha un controllo limitato sul lato di trasporto. In particolare, lo sviluppatore può scegliere il **protocollo di trasporto** (come TCP o UDP) e può configurare alcuni **parametri di trasporto** come la dimensione del buffer.

Tuttavia, una volta scelto il protocollo di trasporto, il funzionamento effettivo della consegna dei messaggi viene gestito dalla rete, che si occupa delle operazioni a livello di trasporto.

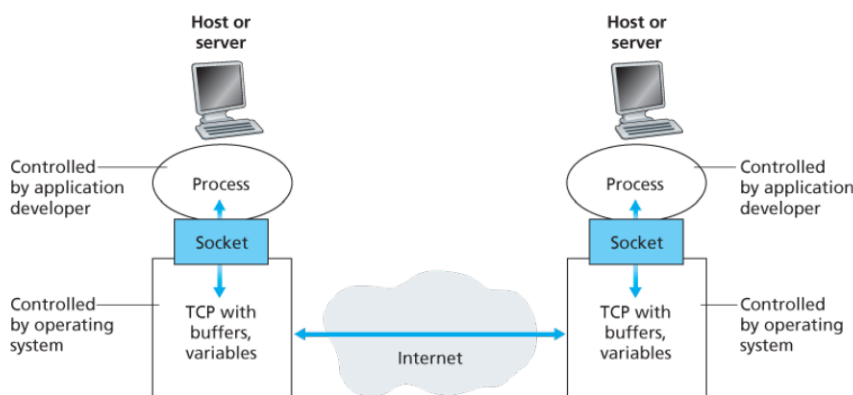


Figure 2.3 Application processes, sockets, and underlying transport protocol

Indirizzamento

Per far sì che un processo su una rete possa ricevere messaggi, è necessario identificarlo in modo univoco.

Ogni dispositivo host ha un **indirizzo IP** (32 o 128 bit), che però da solo non basta, perché sullo stesso host possono essere in esecuzione più processi.

Di conseguenza, l’identificatore completo di un processo include:

- **L’indirizzo IP dell’host** (es. 128.119.245.12 per un server web).
- **Un numero di porta** che specifica il processo destinatario. Alle applicazioni più note sono stati assegnati numeri di porta specifici (es. porta 80 per HTTP).

Gli interlocutori, quindi, memorizzano questa coppia **indirizzo IP + porta** della controparte in un **indirizzo socket**.

Oltre a conoscere l’indirizzo IP dell’**host** destinatario, il mittente deve identificare anche il **processo destinatario** a cui deve inviare il dato. Questo è necessario perché, su un singolo host, possono esserci **molte applicazioni di rete** in esecuzione contemporaneamente, ciascuna che potrebbe utilizzare un **socket** per la comunicazione.

Indirizzamento nelle comunicazioni tra Processi: Well known Ports, Registered Ports e User Ports

Le porte Internet sono utilizzate per distinguere diverse applicazioni o servizi che operano sullo stesso indirizzo IP. I numeri di porta sono suddivisi in tre gruppi principali:

- **Well Known Ports (0 - 1023)**: Riservate a **servizi di sistema** e protocolli noti. Solo i processi con privilegi di **amministratore** possono usarle, garantendo maggiore sicurezza. (es. porta 80 per HTTP, 443 per HTTPS, 25 per SMTP).
- **Registered Ports (1024 - 49151)**: Assegnate da **IANA** per applicazioni specifiche, ma non richiedono privilegi amministrativi, rendendole meno sicure.
- **Dynamic/Private Ports (49152 - 65535)**: Utilizzate per connessioni temporanee, ad esempio dai client per comunicare con i server.

Sicurezza e Affidabilità delle Porte

La distinzione tra **Well Known Ports** e **Registered Ports** è fondamentale per la sicurezza delle comunicazioni. Le prime, essendo accessibili solo agli amministratori, sono generalmente più affidabili. Se un sito bancario utilizza la **porta 8080** anziché la **80 o 443**, potrebbe essere un segnale di configurazione non sicura, poiché chiunque potrebbe aver avviato il servizio senza autorizzazioni specifiche.

Protocolli come **FTP**, e **TELNET** richiedono credenziali di accesso per operare sulle **Well Known Ports**, mentre i servizi sulle **Registered Ports** non necessitano di privilegi particolari, risultando più vulnerabili a potenziali attacchi. Tuttavia, il solo utilizzo di porte riservate non garantisce sempre la sicurezza: un attaccante potrebbe replicare la pagina di login di un sito legittimo e ospitarla su un server con una porta affidabile per ingannare gli utenti.

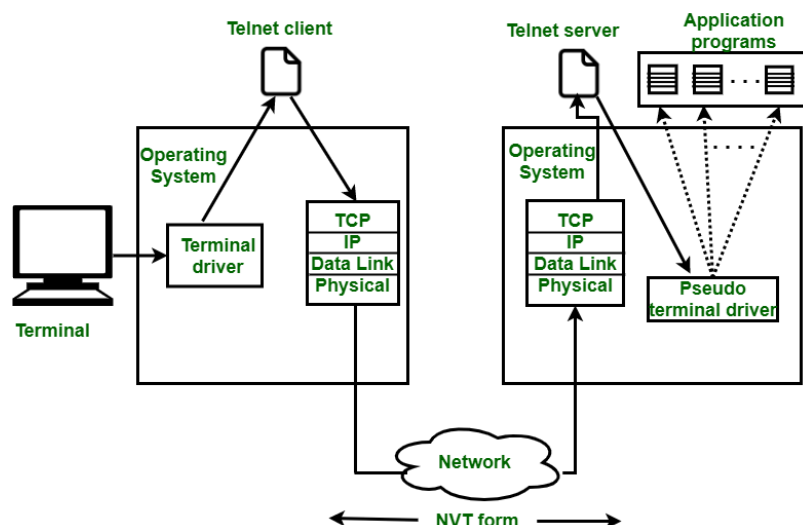
Per verificare l'autenticità di un sito web, si utilizzano i **certificati SSL(Secure Sockets Layer)**, rilasciati da autorità riconosciute. Se un browser rileva un certificato **auto-firmato** o non proveniente da un'ente certificatore ufficiale, avvisa l'utente di un possibile rischio. Spetta all'utente valutare se il sito è affidabile, controllando dettagli come il dominio, l'origine del certificato e la presenza di connessioni sicure.

Telnet

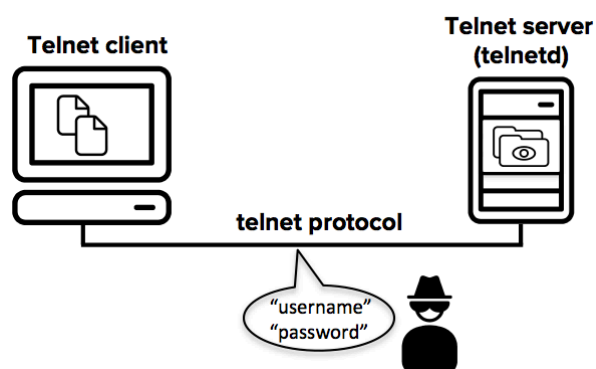
Telnet (Teletype Network) è un protocollo applicativo, comunemente utilizzato su Internet e reti basate su protocolli TCP/IP, che consente di accedere e gestire un computer remoto attraverso una connessione di tipo client-server. Il protocollo è utilizzato principalmente per emulare un terminale, permettendo agli utenti di interagire con un dispositivo remoto e di eseguire comandi come se fossero fisicamente presenti davanti al terminale.

- **Client-Server**: Il client avvia la connessione, mentre il server ascolta le richieste in arrivo e risponde ai comandi inviati dal client.

- **Porta 23:** La porta 23 è riservata per Telnet e viene utilizzata per stabilire la connessione con il dispositivo remoto.



Poiché Telnet non utilizza alcuna forma di crittografia, le informazioni inviate, incluse credenziali di login e informazioni riservate, sono visibili a chiunque abbia accesso alla rete!



Protocollo FTP (File Transfert Protocol)

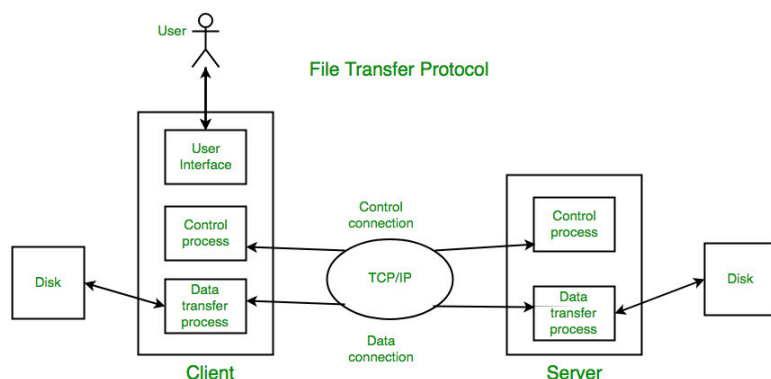
Il **File Transfer Protocol (FTP)** è un protocollo di trasferimento file basato su un'architettura **client-server**. Utilizza due connessioni **TCP** separate: una sulla **porta 21** per il controllo e una sulla **porta 20** per il trasferimento dei dati, permettendo una comunicazione più efficiente.

FTP supporta due modalità di connessione: **attiva** (dove il server si connette al client per il trasferimento dati) e **passiva** (dove il client avvia entrambe le connessioni, utile per superare firewall e NAT).

Una delle sue principali limitazioni è la mancanza di cifratura per dati e credenziali, rendendolo vulnerabile a intercettazioni. Per questo sono stati sviluppati protocolli più sicuri come **FTPS** (che aggiunge SSL/TLS) e **SFTP** (basato su SSH).

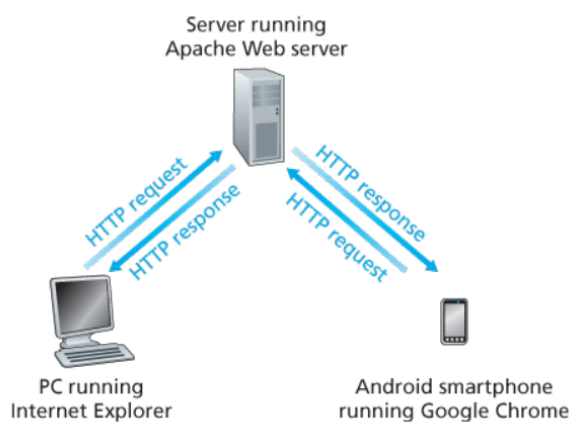
- A differenza di HTTP, FTP richiede autenticazione e mantiene una connessione persistente (fino a quando non viene chiusa dall'utente o fino al verificarsi di un timeout)

per consentire trasferimenti **bidirezionali**, ovvero sia il client che il server possono inviare e ricevere file.



HTTP

Il **protocollo HTTP (HyperText Transfer Protocol)** definisce il modo in cui i client Web richiedono pagine ai server e come questi ultimi rispondono inviando le risorse richieste. È alla base del funzionamento del **World Wide Web**, consentendo la comunicazione tra browser e server attraverso un sistema di richieste e risposte.



Struttura di una Pagina Web

Una pagina web è composta da diversi **oggetti**, ciascuno accessibile tramite un URL separato. Ad esempio:

1. **File HTML base** (es. `index.html`).
2. **Risorse incorporate**, come:
 - Immagini (`.jpg` , `.png` , ecc), video
 - Fogli di stile CSS, script JavaScript

Se una pagina ad esempio contiene 1 file HTML e 5 immagini, il totale degli oggetti richiesti sarà **6**.

Il browser effettua richieste HTTP separate per ogni oggetto necessario alla visualizzazione della pagina.

Modello Client-Server

HTTP segue un **modello client-server**:

- Un **browser** (client) invia richieste HTTP a un server.
- Il **server** elabora la richiesta e restituisce le risorse necessarie, come file HTML, immagini, video, fogli di stile CSS e script JavaScript.
- Ogni risorsa ha un proprio **URL (Uniform Resource Locator)**, che permette di identificarla univocamente su Internet.

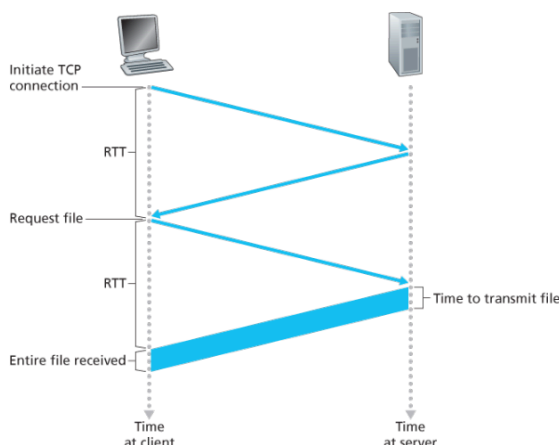
Funzionamento di HTTP

HTTP utilizza il **protocollo TCP (Transmission Control Protocol)** per il trasporto dei dati, garantendo:

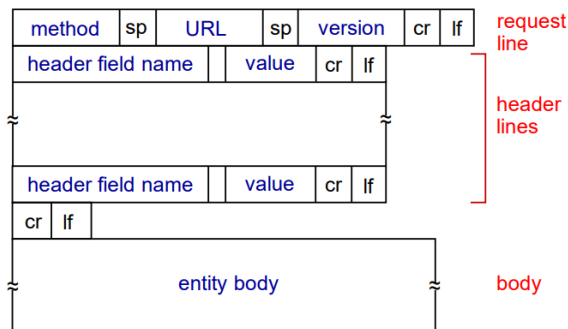
- **Affidabilità**: i dati inviati vengono ricevuti senza errori.
- **Ordinamento**: i pacchetti di dati arrivano nell'ordine corretto.

Fasi della Comunicazione

1. Il **client** avvia una connessione TCP (creando la socket) con il server sulla porta **80** (o **443** per HTTPS).
2. Una volta stabilita la connessione, il client invia una **richiesta HTTP**.
3. Il **server** elabora la richiesta e invia una **risposta HTTP**, contenente la risorsa richiesta o un messaggio di errore.
4. Se viene usata una connessione non persistente (HTTP/1.0), il server chiude la connessione. Con HTTP/1.1 e versioni successive, la connessione può rimanere aperta per ridurre la latenza.

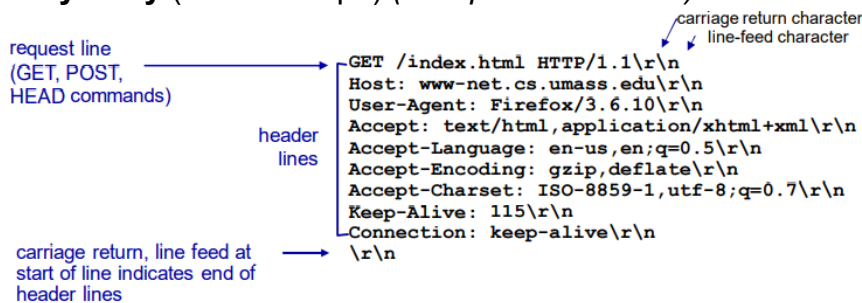


Messaggio di richiesta HTTP



Una richiesta HTTP ha tre sezioni:

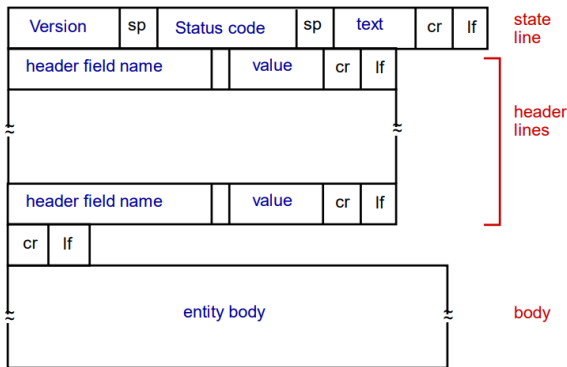
1. **Request line** (prima riga): indica il metodo, l'URL e la versione del protocollo.
 - Esempio: GET /index.html HTTP/1.1 .
2. **Header lines** (seconda riga): contengono informazioni come:
 - Host: → specifica il server destinatario.
 - Connection: close → il client chiede di chiudere la connessione TCP dopo la risposta.
 - User-Agent: → indica il browser o client usato.
 - Accept-Language: → preferenza per la lingua della risposta.
 - Accept: → tipi di contenuto accettati.
 - Referer: → URL della pagina di origine.
3. **Entity body** (ultimo campo) (solo per POST/PUT): contiene i dati inviati dal client.



Metodi HTTP principali

- **GET** → richiede un oggetto.
- **HEAD** → ottiene solo le intestazioni della risposta.
- **POST** → invia dati nel corpo della richiesta.
- **PUT** → carica o sostituisce un file sul server.
- **DELETE** → elimina un oggetto dal server.

Messaggio di risposta HTTP



Una richiesta HTTP ha tre sezioni:

1. **Request line:** indica il metodo, l'URL e la versione del protocollo.
 - Esempio: `GET /index.html HTTP/1.1` → richiesta della risorsa `/index.html`.
2. **Header lines:** contengono informazioni come:
 - `Host`: → specifica il server destinatario.
 - `Connection: close` → il client chiede di chiudere la connessione TCP dopo la risposta.
 - `User-Agent`: → indica il browser o client usato.
 - `Accept-Language`: → preferenza per la lingua della risposta.
 - `Accept`: → tipi di contenuto accettati.
 - `Referer`: → URL della pagina di origine.
 - `Content-Length`: e `Content-Type`: (solo per **POST/PUT**) → dimensione e formato dei dati inviati.
3. **Entity body (solo per POST/PUT):** contiene i dati inviati dal client, come i valori di un modulo.

```

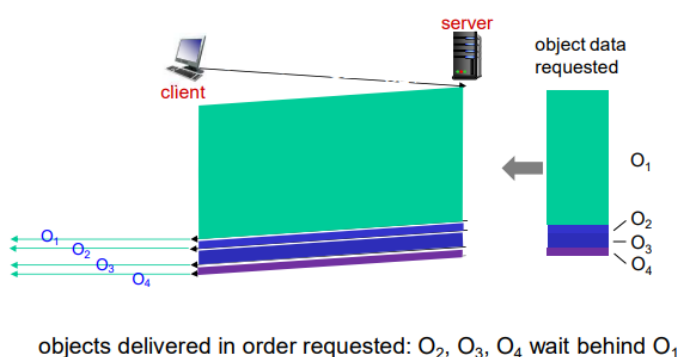
status line (protocol status code status phrase) → HTTP/1.1 200 OK\r\n
                                                    Date: Sun, 26 Sep 2010 20:09:20
                                                    GMT\r\n
                                                    Server: Apache/2.0.52 (CentOS)\r\n
                                                    Last-Modified: Tue, 30 Oct 2007
                                                    17:00:02 GMT\r\n
                                                    ETag: "17dc6-a5c-bf716880"\r\n
                                                    Accept-Ranges: bytes\r\n
                                                    Content-Length: 2652\r\n
                                                    Keep-Alive: timeout=10, max=100\r\n
                                                    Connection: Keep-Alive\r\n
                                                    Content-Type: text/html; charset=ISO-
                                                    8859-1\r\n
                                                    \r\n
data, e.g., requested HTML file → data data data data data ...
    
```

Codici di stato comuni

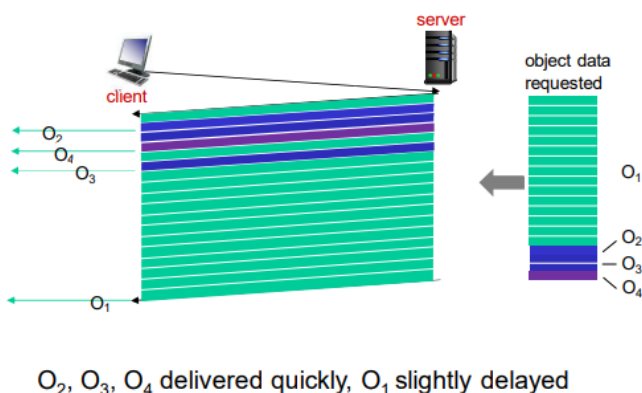
- **200 OK** → Richiesta riuscita, l'oggetto è incluso nella risposta.
- **301 Moved Permanently** → L'oggetto è stato spostato, il nuovo URL è in `Location`.
- **400 Bad Request** → Richiesta non valida.
- **404 Not Found** → L'oggetto richiesto non esiste.
- **505 HTTP Version Not Supported** → Versione HTTP non supportata dal server.

Versioni di HTTP:

- **HTTP/1.0:** - Ogni oggetto richiesto apre una **nuova connessione TCP**. Dopo la trasmissione, la connessione viene **chiusa**. Questo approccio diventa inefficiente con pagine web complesse e ricche di contenuti (richiedono molte connessioni).
- **HTTP/1.1:** Introduce la **connessione TCP persistente**: è possibile inviare più richieste attraverso la **stessa connessione**, riducendo l'overhead. Le richieste vengono elaborate in **ordine di arrivo (FCFS)**. Se il primo oggetto richiesto è grande (es. un video), gli oggetti più piccoli in coda **devono aspettare** → si verifica il cosiddetto **head-of-line (HOL) blocking**. Se un segmento TCP si perde, **tutta la connessione si blocca** fino alla ritrasmissione.



- **HTTP/2:** **Connessione unica TCP** per tutte le risorse di una pagina. Gli oggetti sono suddivisi in **frame**, che possono essere **interlacciati** e inviati in ordine diverso. Supporta **priorità delle richieste** e la **Compressione delle intestazioni HTTP** → tuttavia non vi è alcuna **cifratura**: per avere sicurezza, si deve usare **HTTPS** (HTTP/2 su TLS).



- **HTTPS** (HyperText Transfer Protocol Secure) è la versione sicura di HTTP e garantisce che i dati scambiati tra browser e server siano **criptati** e protetti da intercettazioni. Utilizza **TLS/SSL** (Transport Layer Security / Secure Sockets Layer)
- **HTTP/3:** supera i limiti di TCP utilizzando il **protocollo QUIC** (TCP + TLS combinati in un **unico protocollo basato su UDP**), introducendo **Cifratura integrata** di default, **controllo di errore e congestione per flusso**, quindi la perdita di pacchetti **non blocca tutte le risorse** e **Pipelining più efficiente**, senza head-of-line blocking a livello di trasporto.

HTTP e la gestione delle sessioni

HTTP è un **protocollo senza stato (stateless)**: ogni volta che un client (come un browser web) invia una richiesta, il server la tratta come una nuova richiesta indipendente, senza

alcun ricordo delle interazioni passate.

Questo approccio semplifica il protocollo, ma crea difficoltà quando è necessario tenere traccia di sessioni o autenticazioni degli utenti, per cui si usano tecniche come i **cookie**.

Cookies

I cookie sono un meccanismo utilizzato dai server web per mantenere traccia degli utenti nonostante HTTP sia un protocollo **stateless** (senza memoria delle richieste precedenti). Consentono a un sito web di riconoscere un utente tra diverse sessioni, migliorando l'esperienza d'uso ma sollevando anche preoccupazioni sulla privacy.

Il **file cookie** è un semplice **file di testo** salvato dal browser sul dispositivo dell'utente.

I cookies si basano su quattro **elementi principali**:

1. Un'intestazione Set-Cookie nella risposta HTTP del server (chiede al browser di **salvare il cookie** e di inviarlo nelle future richieste.)
2. Un'intestazione Cookie nelle richieste HTTP del browser (una volta che il browser ha memorizzato il cookie, lo invia automaticamente in tutte le richieste HTTP successive verso lo stesso server)
3. Un file cookie memorizzato nel sistema dell'utente e gestito dal browser.
4. Un database sul server che associa il cookie a un identificativo univoco dell'utente (il server mantiene un **database** che associa il valore del cookie (es. `sessionId=abc123`) a un **identificativo univoco** dell'utente.)

Come Funzionano i Cookie?

1. Prima visita al sito:

- Il server assegna all'utente un **ID univoco** (es. `1678`) e lo inserisce in un database.
- Invia una risposta HTTP con l'header `Set-Cookie: ID`.
- Il browser salva l'ID in un **file dei cookie**, associandolo al dominio del sito (es. Amazon).

2. Visite successive:

- Il browser invia automaticamente l'ID nell'header `Cookie: ID` con ogni richiesta al sito.
- Il server riconosce l'utente e personalizza l'esperienza (es. carrello della spesa, suggerimenti).

3. Autenticazione persistente:

- Se l'utente si registra, il sito collega l'ID del cookie alle sue informazioni personali (nome, indirizzo, ecc.).
- Il server valida la richiesta e genera un **cookie di sessione** (numero casuale).
- Questo cookie viene incluso in tutte le richieste successive, mantenendo la sessione aperta.

4. Sicurezza e vulnerabilità:

- Il cookie viene memorizzato nell'**User-Agent** fino alla chiusura della sessione o alla sua scadenza.
- Alcuni siti **mantengono il cookie**, lasciando aperta la sessione di autenticazione.
- Se un malintenzionato ruba questo **cookie di sessione**, può accedere al sito come utente autenticato.

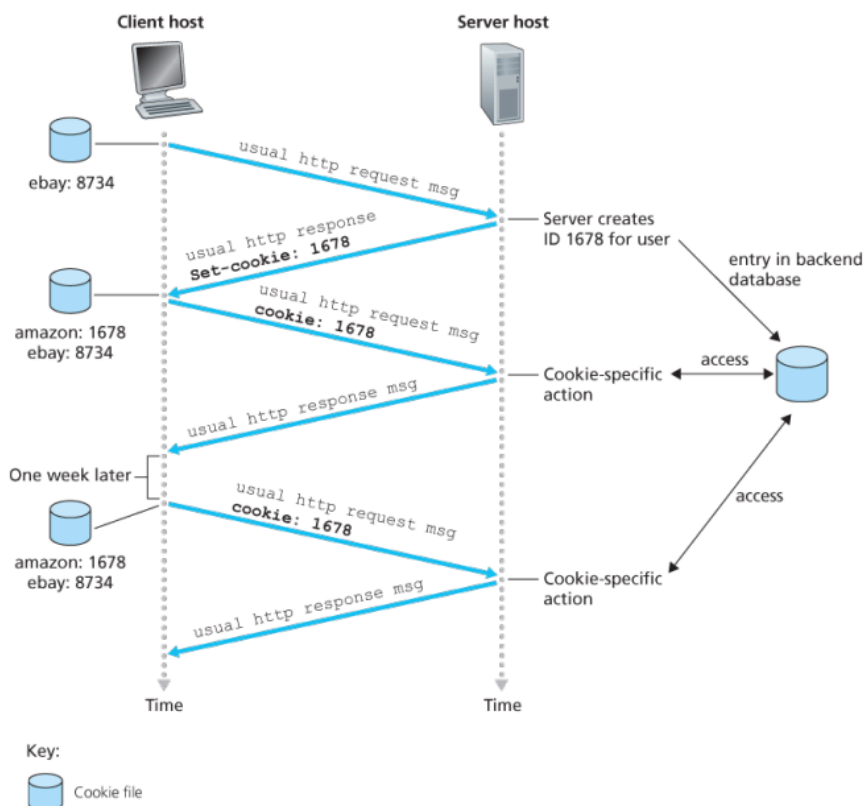


Figure 2.10 Keeping user state with cookies

Utilizzi dei Cookie

Autenticazione (mantenere l'accesso a servizi); conservare gli articoli selezionati; **raccomandazioni personalizzate**; **tracciamento comportamentale** sia su un singolo sito (cookie di prima parte) che su più siti (cookie di terze parti).

Cookies di terze parti

I **cookie persistenti di terze parti** (o **tracking cookies**) sono cookie impostati da un dominio diverso da quello del sito che l'utente sta visitando. A differenza dei **cookie di prima parte**, utilizzati per funzionalità essenziali come l'autenticazione, questi servono principalmente per **tracciare la navigazione dell'utente su più siti web** e creare un profilo dettagliato del suo comportamento online.

Il tracciamento non avviene solo tramite banner pubblicitari visibili, ma anche attraverso **pixel invisibili** o **link nascosti**, che attivano richieste HTTP ai server di tracciamento. Quando l'utente visita altri siti che integrano gli stessi servizi, il cookie viene nuovamente inviato, permettendo di monitorare i suoi movimenti online e di offrire pubblicità personalizzata, spesso senza che l'utente ne sia consapevole.

Il **GDPR** (Regolamento Generale sulla Protezione dei Dati dell'UE) considera i cookie come **dati personali** se possono identificare un individuo. Questo significa che:

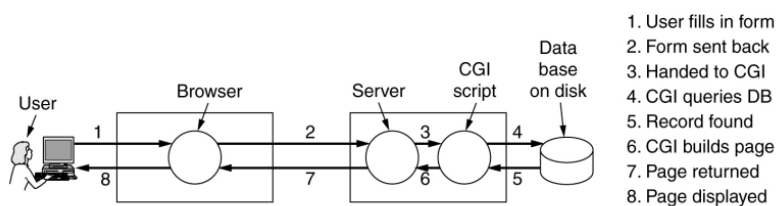
- I siti web devono **chiedere il consenso esplicito** dell'utente prima di memorizzare o leggere i cookie non essenziali (ad es. quelli per il tracciamento o la pubblicità).
- L'utente deve avere **controllo totale** sui cookie, con la possibilità di accettarli, rifiutarli o modificarne le preferenze.

DHTML

Il **DHTML (Dynamic HTML)** è un insieme di tecnologie che combinano **HTML, CSS e JavaScript** per rendere le pagine web dinamiche e interattive. Non è un linguaggio a sé, ma un metodo per modificare contenuti e stile senza ricaricare la pagina.

Un utente interagisce con il **browser**, che comunica con il **server**. Quest'ultimo può eseguire script dinamici, basati su un database, per generare la pagina da restituire al browser. Inizialmente, questi script erano scritti in **C**, ma per motivi di sicurezza sono stati sostituiti da linguaggi di **scripting** come **PHP, ASP e Python**.

Oggi, il DHTML si è evoluto con tecnologie come **AJAX** e framework moderni (React, Angular, Vue.js), rendendo il web più interattivo.



Proxy server / Web Cache

Il **Web caching**, conosciuto anche come **proxy caching**, è una tecnica fondamentale utilizzata per migliorare le prestazioni della rete e ridurre la congestione del traffico Internet. Si tratta di un meccanismo attraverso cui un'entità di rete, chiamata **Web cache** o **proxy server**, si interpone tra i browser degli utenti e i server Web originari, salvando copie locali degli oggetti richiesti (come pagine HTML, immagini, file multimediali, ecc.) in uno spazio di memoria (solitamente su disco).

Come funziona una Web Cache?

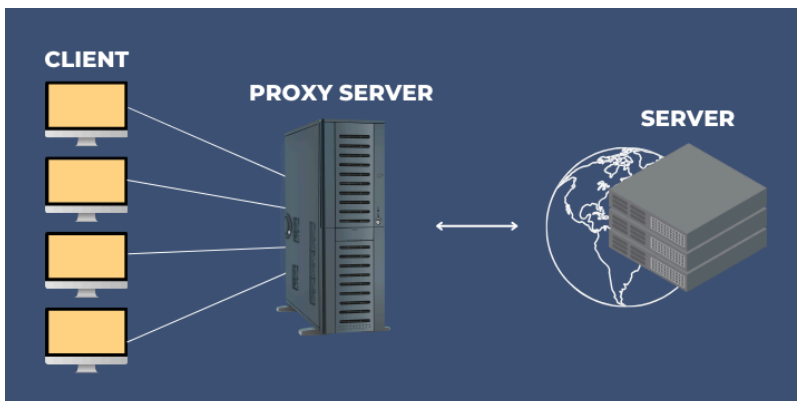
Una Web cache (o Proxy Server) può essere installata da un provider Internet (ISP) o da un'organizzazione (es. un'università) all'interno della propria rete. Una volta configurato il browser dell'utente per comunicare con il proxy, ogni richiesta HTTP passa attraverso la cache:

1. Il browser stabilisce una **connessione TCP** con il Web cache e invia una **richiesta HTTP** per l'oggetto desiderato.
2. Il Web cache **controlla se possiede già una copia** dell'oggetto nella propria memoria locale:
 - Se l'oggetto è presente (**cache hit**), lo invia direttamente al browser tramite una risposta HTTP.
 - Se l'oggetto è assente (**cache miss**), il Web cache apre una nuova connessione TCP al **server originario** e inoltra la richiesta.
3. Il server originario invia l'oggetto al cache, che:
 - lo **memorizza localmente**, per future richieste simili,
 - e lo **inoltra al client**.

La Web cache agisce **come server nei confronti del browser** e **come client nei confronti del server Web originario**.

Il Web caching è una **tecnologia chiave** per garantire:

- Velocità di caricamento;
- Efficienza di rete,
- Risparmio economico.



Il Conditional GET e la Coerenza della Cache

Sebbene la cache Web sia un'ottima soluzione per alleggerire il traffico di rete, introduce una nuova problematica: **il rischio che la copia dell'oggetto salvata nella cache sia obsoleta**. Il meccanismo del **Conditional GET** serve a garantire che ciò non accada.

Quando un oggetto viene salvato nella cache, viene memorizzata anche la data della sua ultima modifica.

Se in seguito un altro utente richiede lo stesso oggetto, il proxy può inviare una richiesta HTTP **condizionale**, usando l'header `If-Modified-Since`. Il server Web controllerà se l'oggetto è stato modificato dopo quella data:

- Se **non è stato modificato**, risponde con `304 Not Modified`, evitando di reinviare l'oggetto (risparmio di banda).

- Se è **stato modificato**, invia la versione aggiornata.

SMTP, POP3, IMAP4 e il confronto con HTTP

La gestione della posta elettronica si basa su protocolli distinti per l'invio e la ricezione dei messaggi. I principali sono **SMTP**, **POP3** e **IMAP4**, ciascuno con caratteristiche e funzioni specifiche.

SMTP (Simple Mail Transfer Protocol)

SMTP (Simple Mail Transfer Protocol) è il protocollo standard per l'**invio di email**, utilizzato sia dai client di posta sia dai server per trasmettere i messaggi. Si basa su un meccanismo **push**, in cui il mittente avvia la connessione e **invia attivamente** il messaggio al server del destinatario, tramite una comunicazione **riga per riga** con caratteri **ASCII a 7 bit** (fino a 127 caratteri leggibili).

In origine, le email erano semplici **file di testo**, e inviarne una significava aggiungere una sezione di testo alla fine del file di posta della macchina destinataria. Un dettaglio tecnico interessante è l'uso del **punto (.)** come indicatore di comandi speciali, poiché non viene mai usato all'inizio di una normale riga di testo.

Per superare i limiti legati all'invio di contenuti non testuali (es. immagini, video, documenti), è stato introdotto **MIME (Multipurpose Internet Mail Extensions)**, un sistema che consente di convertire i dati binari in formato testuale ASCII, rendendoli compatibili con SMTP.

Funzionamento

1. L'utente scrive e invia un messaggio tramite un **User Agent** (come Outlook o Gmail).
2. Il server SMTP del mittente apre una **connessione TCP** (solitamente sulla porta 25) con il server destinatario.
3. Dopo l'**handshake iniziale** (HELO , MAIL FROM , RCPT TO), il messaggio viene trasmesso **riga per riga**.
4. Il server destinatario riceve il messaggio e lo deposita nella **mailbox** del destinatario.

Limitazioni

- SMTP **non gestisce la ricezione dell'email da parte dell'utente finale**; si occupa solo del **trasferimento tra server**.
- Per l'accesso alla posta ricevuta, è necessario ricorrere a protocolli aggiuntivi come **POP3** o **IMAP**.

POP3 (Post Office Protocol v3)

POP3 (Post Office Protocol v3) è un protocollo di tipo **pull** utilizzato per **ricevere la posta elettronica**. Il client si connette al server, invia le **credenziali di accesso** (username e password) e **scarica interamente i messaggi** disponibili sul dispositivo locale. A seconda

della configurazione, i messaggi possono essere cancellati dal server dopo il download oppure conservati.

POP3 è pensato per un utilizzo **da un singolo dispositivo**, poiché **non sincronizza lo stato dei messaggi** (letti, non letti, eliminati) e **non supporta la gestione remota delle cartelle**. Questo lo rende inadatto per l'accesso alla posta da più dispositivi.

Funzionamento

1. **Autorizzazione:** Il client invia username e password (in chiaro, se non si usa SSL/TLS).
2. **Transazione:** Il client può:
 - Listare i messaggi (`LIST`).
 - Scaricarli (`RETR`).
 - Marcarli per la cancellazione (`DELE`).
3. **Aggiornamento:** Al comando `QUIT` , i messaggi contrassegnati vengono rimossi dal server.

Problemi

- **Scarica e cancella:** Se attivato, rende impossibile rileggere la posta da altri dispositivi.
- **Nessuna sincronizzazione:** Lo stato (es. "letta/non letta") non viene mantenuto tra sessioni.
- **Nessuna gestione di cartelle remote:** Le email possono essere organizzate solo localmente.
- **Trasmette le credenziali in chiaro**, per cui è necessaria una connessione cifrata tramite **SSL/TLS** per garantire la sicurezza.

IMAP (Internet Message Access Protocol)

IMAP4 (Internet Message Access Protocol versione 4) è un protocollo per ricevere e gestire la posta elettronica, nato per superare le limitazioni del più semplice POP3. A differenza di quest'ultimo, che scarica le email sul dispositivo e poi, spesso, le elimina dal server, **IMAP consente di mantenere i messaggi sul server** e accedervi in qualsiasi momento da diversi dispositivi, mantenendo sincronizzato lo stato della casella di posta.

Il funzionamento di IMAP è orientato a un uso **multi-dispositivo**: i messaggi non vengono più trasferiti e archiviati localmente, ma **restano nel server di posta**. In questo modo, se si legge o si elimina un'email da un computer, lo stesso cambiamento sarà visibile anche accedendo da un altro dispositivo. Questo approccio risolve uno dei problemi principali del POP3, che non è in grado di sincronizzare tra diversi client e non permette la gestione remota delle cartelle.

Infatti, con IMAP l'utente può **organizzare la posta direttamente sul server**, creando nuove cartelle, spostando i messaggi tra esse o cancellandoli, e tutte queste modifiche restano valide indipendentemente dal dispositivo utilizzato.

Un altro aspetto interessante è la possibilità di **accedere solo a una parte del messaggio**: ad esempio, si può visualizzare soltanto l'intestazione (mittente, oggetto, data) senza scaricare l'intero contenuto.

Inoltre, IMAP permette di **cercare le email direttamente sul server**, senza doverle prima scaricare sul proprio dispositivo.

Problemi

- Come POP3, anche IMAP **invia le credenziali (login e password) in chiaro**, quindi è fondamentale **utilizzare una connessione protetta**, ad esempio tramite SSL/TLS, per garantire la sicurezza dei dati durante lo scambio.
- **Maggior carico sul server**: Poiché tutti i messaggi e le cartelle rimangono sul server, IMAP richiede più spazio di archiviazione e risorse rispetto a POP3.

Confronto tra SMTP e HTTP

Sebbene SMTP e HTTP siano entrambi protocolli per il trasferimento di dati, esistono alcune differenze fondamentali nel modo in cui operano.

1. Direzione del trasferimento:

- **SMTP** è un protocollo **push**: il server mittente avvia la connessione e invia il messaggio al server destinatario.
- **HTTP** è un protocollo **pull**: il client (di solito un browser) avvia la connessione per richiedere un contenuto da un server.

2. Formato dei dati:

- **SMTP** supporta solo **caratteri ASCII a 7 bit**, quindi per inviare contenuti non testuali serve una codifica (es. MIME).
- **HTTP** non ha questo limite e può trasmettere direttamente **dati binari o codificati in altri formati**.

3. Gestione dei contenuti multimediali:

- **SMTP** inserisce **tutti gli oggetti** (testo, immagini, allegati) **in un unico messaggio**.
- **HTTP** trasmette **ogni oggetto separatamente**, incapsulato in una propria risposta.

//Livello di trasporto

-Livello end to end

I messaggi devono essere di lunghezza prestabilita, non arbitraria

Il livello di trasporto (Transport Layer) fornisce una comunicazione logica tra i processi applicativi in esecuzione su host diversi.

Tutti i protocolli di trasporto funzionano in modo **end-to-end**, ovvero:

- **Mittente:** suddivide i messaggi dell'applicazione in segmenti e li passa al livello di rete. A seconda dell'ambito, usare grandezze dinamiche o variabili .
- **Destinatario:** riassembla i segmenti per ricostruire il messaggio originale e lo passa al livello applicativo.

La suite **TCP/IP** offre due protocolli di trasporto per le applicazioni Internet:

- **TCP (Transmission Control Protocol):** affidabile, orientato alla connessione, garantisce l'ordine e l'integrità dei dati.
- **UDP (User Datagram Protocol):** più veloce, non orientato alla connessione, non garantisce l'ordine o la consegna dei pacchetti.

Il livello di trasporto **isola i livelli superiori dai dettagli della trasmissione dati**, fornendo servizi di comunicazione indipendenti dalla tecnologia della rete sottostante.

mtu --> grandezza massima del frame

```
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP qlen 1000
```

su ipv6 non si frammenta mai l'header, su ipv4 si